

Makine Öğrenmesi

Matematikten Koda Kapsamlı Rehber

Teori • Türetmeler • Sıfırdan ve scikit-learn ile Uygulama

Bu belge, makine öğrenmesini **matematiksel temellerinden başlayarak** her algoritmanın türetmesiyle, hem **sıfırdan NumPy** hem de **scikit-learn** uygulamalarıyla adım adım anlatır. Lineer cebir ve optimizasyondan derin öğrenmeye ve geri yayılıma kadar uzanır.

Kısım 1 — Matematiksel Temeller: lineer cebir • kalkülüs & gradyanlar • optimizasyon (gradient descent) • olasılık & MLE • bilgi teorisi.

Kısım 2 — Öğrenme Teorisi: kayıp fonksiyonları • risk • bias-variance • düzenleme • değerlendirme.

Kısım 3 — Regresyon: lineer regresyon (normal denklem + GD) • Ridge/Lasso • lojistik regresyon.

Kısım 4 — Sınıflandırma: Naive Bayes • KNN • SVM (marj + kernel) • karar ağaçları • ensemble & boosting.

Kısım 5 — Denetimsiz: K-Means • PCA (özdeğer türetmesi).

Kısım 6 — Derin Öğrenme: perceptron • çok katmanlı ağlar • geri yayılım (tam türetme) • optimizasyon • CNN & RNN.

Hazırlanma Tarihi: 12 Temmuz 2026

Python 3 • NumPy • scikit-learn • Kodlar İngilizce, açıklamalar Türkçe

Contents

1	Lineer Cebir	2
1.1	Skaler, Vektör, Matris, Tensör	2
1.2	İç Çarpım (Dot Product)	2
1.3	Norm: Vektör Uzunluğu	3
1.4	Kosinüs Benzerliği ve Uzaklık Ölçütleri	3
1.5	Matris Çarpımı	4
1.6	Şekil Uyumu ve Yayınlama (Broadcasting)	4
1.7	Özel Matrisler ve İşlemler	5
1.8	Özdeğerler ve Özvektörler	5
2	Kalkülüs ve Optimizasyon	6
2.1	Türev: Değişim Oranı	6
2.2	Kısmi Türev ve Gradyan	6
2.3	Zincir Kuralı: Backprop'un Kalbi	6
2.4	Kısmi Türev ve Gradyanın Somut Hesabı	7
2.5	Gradient Descent (Gradyan İnişi)	7
2.6	Gradient Descent Türleri	8
2.7	Konvekslik: Neden Bazı Problemler Kolaydır?	8
3	Olasılık ve İstatistik	9
3.1	Temel Kavramlar	9
3.2	Önemli Olasılık Dağılımları	9
3.3	Bağımsızlık ve Olasılık Zincir Kuralı	10
3.4	Koşullu Olasılık ve Bayes Teoremi	10
3.5	Maksimum Olabilirlik Tahmini (MLE)	10
4	Bilgi Teorisi	11
4.1	Entropi	11
4.2	Çapraz Entropi ve KL İraksaması	12
5	Öğrenme Problemi ve Kayıp Fonksiyonları	14
5.1	Temel Bileşenler	14
5.2	Yaygın Kayıp Fonksiyonları	14
5.3	Ampirik Risk Minimizasyonu	14
6	Aşırı Öğrenme, Bias-Variance ve Düzenleme	15
6.1	Aşırı ve Eksik Öğrenme	15
6.2	Bias-Variance Ayrışması	15
6.3	Düzenleme (Regularization)	15
7	Lineer Regresyon	17
7.1	Model	17
7.2	Maliyet Fonksiyonu	17
7.3	Çözüm 1: Normal Denklem (Kapalı Form)	17
7.4	Çözüm 2: Gradient Descent	18
7.5	scikit-learn ile	19

8 Ridge, Lasso ve Lojistik Regresyon	19
8.1 Ridge Regresyonu (L2)	19
8.2 Neden Lasso Sıfırlar, Ridge Sıfırlamaz?	19
8.3 Lojistik Regresyon: Sınıflandırmaya Köprü	20
8.4 Kayıp: İkili Çapraz Entropi (MLE'den)	20
9 Naive Bayes ve K-En Yakın Komşu	22
9.1 Naive Bayes	22
9.2 K-En Yakın Komşu (KNN)	23
10 Destek Vektör Makineleri (SVM)	23
10.1 Maksimum Marj	23
10.2 Yumuşak Marj (Soft Margin)	24
10.3 Kernel Hilesi (Kernel Trick)	24
11 Karar Ağaçları	24
11.1 Bölme Kriteri: Bilgi Kazancı ve Gini	25
12 Topluluk Yöntemleri (Ensemble)	25
12.1 Bagging ve Random Forest	25
12.2 Boosting: Hataları Ardışık Düzeltmek	26
13 K-Means Kümeleme	27
13.1 Amaç Fonksiyonu	27
13.2 Lloyd Algoritması	27
14 Temel Bileşen Analizi (PCA)	28
14.1 Matematiksel Türetme	28
15 Perceptron'dan Çok Katmanlı Ağlara	30
15.1 Yapay Nöron	30
15.2 Aktivasyon Fonksiyonları	30
15.3 İleri Besleme (Forward Propagation)	31
16 Geri Yayılım (Backpropagation)	31
16.1 Neden Zincir Kuralı?	31
16.2 Tam Türetme (Softmax + Çapraz Entropi)	31
16.3 Kaybolan/Patlayan Gradyan	33
17 Optimizasyon Algoritmaları	33
17.1 Momentum	33
17.2 Adam: En Yaygın Optimize Edici	33
18 Derin Öğrenme Mimarileri ve Pratik	33
18.1 Evrişimli Ağlar (CNN)	34
18.2 Tekrarlayan Ağlar (RNN) ve Transformer	34
19 Kapanış: Bir Bütün Olarak Makine Öğrenmesi	34
19.1 Bir Model Nasıl Seçilir?	35

KISIM 1

Matematiksel Temeller

Makine öğrenmesi, özünde *matematiktir*: veriyi vektör ve matrislerle temsil eder (lineer cebir), bir hata fonksiyonunu türev alarak küçültür (kalkülüs & optimizasyon) ve belirsizliği olasılıkla modeller. Bu kısım, sonraki tüm algoritmaların üzerine kurulacağı matematiksel zemini atar. Aceleyle geçme — buradaki her kavram ileride tekrar tekrar karşına çıkacak.

1. Lineer Cebir

Lineer cebir, makine öğrenmesinin dilidir. Bir veri kümesindeki her örnek bir *vektör*, tüm veri kümesi bir *matris*, bir modelin parametreleri de vektör/matrislerdir. Modern donanım (CPU/GPU) matris işlemlerinde son derece hızlı olduğundan, algoritmaları matris biçiminde ifade etmek hem zarif hem verimlidir.

1.1 Skaler, Vektör, Matris, Tensör

Nesne	Gösterim	Anlamı
Skaler	$x \in \mathbb{R}$	Tek sayı.
Vektör	$\mathbf{x} \in \mathbb{R}^n$	n sayıdan oluşan dizi (bir örnek/nokta).
Matris	$\mathbf{X} \in \mathbb{R}^{m \times n}$	m satır n sütun (m örnek, n özellik).
Tensör	$\mathcal{X}$	3+ boyutlu dizi (görüntü, video).

Bir makine öğrenmesi problemi genelde *tasarım matrisi* $\mathbf{X} \in \mathbb{R}^{m \times n}$ ile ifade edilir: her satır bir örnek (m tane), her sütun bir özellik (n tane). Hedef değerler ise $\mathbf{y} \in \mathbb{R}^m$ vektöründedir.

1.2 İç Çarpım (Dot Product)

İki vektörün iç çarpımı, makine öğrenmesindeki en temel işlemdir: bir modelin “ağırlıklı toplam” tahmini tam olarak budur. Tanım:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n.$$

İç çarpım aynı zamanda vektörler arası açıyla ilişkilidir: $\mathbf{a}^\top \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$. Bu yüzden iç çarpım bir *benzerlik* ölçüsüdür: aynı yöne bakan vektörlerde büyük, dik vektörlerde sıfırdır.

```
1 import numpy as np
```

```
2
```

```

3 a = np.array([1.0, 2.0, 3.0])
4 b = np.array([4.0, 5.0, 6.0])
5
6 dot = a @ b          # 1*4 + 2*5 + 3*6 = 32
7 dot = np.dot(a, b)   # aynı sonuç
8 dot = np.sum(a * b)   # elle: eleman bazlı çarpım sonra toplam

```

1.3 Norm: Vektör Uzunluğu

Norm, bir vektörün “büyüklüğünü” ölçer. Kayıp fonksiyonları ve düzenleştirme normlarla tanımlanır. En yaygın L_2 (Öklid) normudur:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^\top \mathbf{x}}, \quad \|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|.$$

L_1 normu (mutlak değerler toplamı) Lasso düzenleştirmesinde, L_2 normu Ridge’de karşımıza çıkacak. İkisi arasındaki fark, ileride göreceğimiz gibi, modelin davranışını kökten değiştirir.

```

1 x = np.array([3.0, 4.0])
2 l2 = np.linalg.norm(x)          # sqrt(9 + 16) = 5.0
3 l1 = np.linalg.norm(x, ord=1)   # |3| + |4| = 7.0

```

1.4 Kosinüs Benzerliği ve Uzaklık Ölçütleri

İki veri noktasının “ne kadar benzer” veya “ne kadar uzak” olduğunu ölçmek; KNN, k-means, öneri sistemleri ve metin gömme (embedding) gibi birçok algoritmanın temelidir. İki temel yaklaşım vardır: *açı* tabanlı benzerlik ve *uzaklık*.

Kosinüs Benzerliği (Cosine Similarity)

İç çarpımdan türeyen kosinüs benzerliği, iki vektörün *yönlerinin* ne kadar hizalı olduğunu ölçer — büyüklüklerinden bağımsız olarak:

$$\cos \theta = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \in [-1, 1].$$

+1 aynı yön (çok benzer), 0 dik (ilgisiz), -1 zıt yön demektir.

Türetme

Örnek: $\mathbf{a} = (1, 2, 3)$, $\mathbf{b} = (2, 4, 6)$ olsun. Dikkat: $\mathbf{b} = 2\mathbf{a}$, yani aynı yön. İç çarpım $\mathbf{a}^\top \mathbf{b} = 2 + 8 + 18 = 28$; normlar $\|\mathbf{a}\| = \sqrt{14}$, $\|\mathbf{b}\| = \sqrt{56} = 2\sqrt{14}$. O halde:

$$\cos \theta = \frac{28}{\sqrt{14} \cdot 2\sqrt{14}} = \frac{28}{28} = 1.$$

Büyüklükleri farklı olsa da (biri diğerinin iki katı), yönleri aynı olduğu için benzerlik tamdır. Buna karşılık $\mathbf{a} = (1, 0)$ ve $\mathbf{b} = (0, 1)$ için iç çarpım 0 olduğundan $\cos \theta = 0$: dik, yani “ilgisiz”.

Uzaklık Ölçütleri (Distance Metrics)

Uzaklık ise iki noktanın *konum* farkını ölçer. İki yaygın ölçüt:

$$\text{Öklid } (L_2): d_2(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_i (a_i - b_i)^2}, \quad \text{Manhattan } (L_1): d_1 = \sum_i |a_i - b_i|.$$

Örneğin $\mathbf{a} = (1, 2)$, $\mathbf{b} = (4, 6)$ için: $d_2 = \sqrt{9 + 16} = 5$, $d_1 = 3 + 4 = 7$.

```

1 import numpy as np
2
3 a = np.array([1.0, 2.0, 3.0])
4 b = np.array([2.0, 4.0, 6.0])
5
6 cos_sim = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))    # 1.0 (aynı yön)
7 euclidean = np.linalg.norm(a - b)                             # Öklid uzaklığı
8 manhattan = np.sum(np.abs(a - b))                             # Manhattan uzaklığı

```

Bilgi

Benzerlik mi, uzaklık mı? Kosinüs benzerliği büyüklüğü *yok sayar*, yalnızca yöne bakar — metin/embedding karşılaştırmasında ideal (uzun bir belge ile kısa bir belge aynı konudaysa benzer sayılmalı). Öklid uzaklığı ise büyüklüğe *duyarlıdır*; bu yüzden KNN ve k-means'te özellikleri önce standartlaştırmak (aynı ölçeğe getirmek) şarttır.

1.5 Matris Çarpımı

Matris çarpımı, bir veri kümesindeki tüm örnekler için tahminleri *tek seferde* hesaplamamızı sağlar. $\mathbf{A} \in \mathbb{R}^{m \times k}$ ve $\mathbf{B} \in \mathbb{R}^{k \times n}$ için çarpım $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times n}$:

$$C_{ij} = \sum_{l=1}^k A_{il}B_{lj}.$$

Çarpımın tanımlı olması için ilk matrisin sütun sayısı, ikincinin satır sayısına eşit olmalıdır. Bu “iç boyutların eşleşmesi” kuralı, uygulamada boyut hatalarının ana kaynağıdır.

```

1 A = np.array([[1, 2],
2               [3, 4]])    # 2x2
3 B = np.array([[5, 6],
4               [7, 8]])    # 2x2
5
6 C = A @ B                # matris çarpımı
7 # C = [[1*5+2*7, 1*6+2*8],
8 #      [3*5+4*7, 3*6+4*8]] = [[19, 22], [43, 50]]
9
10 # TÜM örnekler için tek seferde bir model tahmini:
11 # X: (m, n) tasarım matrisi, w: (n,) ağırlıklar -> tahminler: (m,)
12 X = np.random.randn(100, 3)    # 100 örnek, 3 özellik
13 w = np.array([0.5, -1.2, 2.0])
14 predictions = X @ w           # (100,) -- her satır için bir iç çarpım

```

1.6 Şekil Uyumu ve Yayınlama (Broadcasting)

Matris/vektör işlemlerinde en sık karşılaşılan hata, *boyut uyumsuzluğudur* (shape mismatch). İki kuralı içselleştirmek, ML kodundaki hataların büyük kısmını önler:

- **Matris çarpımı** $\mathbf{A}_{(m \times k)} \mathbf{B}_{(k \times n)}$: iç boyutlar (k) eşleşmeli; sonuç $(m \times n)$.
- **Yayınlama (broadcasting)**: eleman bazlı işlemlerde NumPy şekilleri *sondan başa* karşılaştırır; her boyut ya eşit ya da biri 1 olmalıdır. Boyutu 1 olan taraf diğerine “yayılır” — bellekte kopya oluşturmadan.

A şekli	B şekli	Sonuç	Durum
(100, 3)	(3,)	(100,)	A @ B: geçerli (iç boyut 3)
(100, 3)	(3,)	(100, 3)	A + B: her satıra yayılır
(100, 1)	(1, 3)	(100, 3)	Dış çarpım gibi yayılır
(100, 3)	(100,)	--	Hata: 3 ile 100 uyumsuz

```

1 import numpy as np
2
3 X = np.random.randn(100, 3)      # 100 örnek, 3 özellik
4
5 # Her sütunu standartlaştır (z-skoru) -- yayınlamanın klasik kullanımı
6 mean = X.mean(axis=0)            # (3,) her özelliğin ortalaması
7 std = X.std(axis=0)              # (3,)
8 X_scaled = (X - mean) / std      # (100,3) - (3,) -> (3,) her satıra yayılır
9
10 # Sık hata: (100,) ile (100,3) toplanamaz -> yeniden şekillendir
11 col = np.arange(100)            # (100,)
12 result = X + col[:, None]       # (100,1) yapılarak her sütuna yayılır

```

Dikkat

Boyut hatası aldığında (ValueError: operands could not be broadcast), önce ilgili dizilerin .shape'ini yazdır. Sorun neredeyse her zaman bir yerde beklenmedik bir eksik/fazla boyuttur. None (veya np.newaxis) ile yeni bir eksen ekleyerek şekilleri uyumlu hale getirebilirsiniz.

1.7 Özel Matrisler ve İşlemler

Kavram	Tanım / önem
Transpoz A^T	Satır ve sütunları takas eder; $(A^T)_{ij} = A_{ji}$.
Birim matris I	Köşegeni 1, gerisi 0; $AI = A$.
Ters matris A^{-1}	$AA^{-1} = I$; sadece kare ve tekil olmayan matrisler için.
Determinant $\det(A)$	0 ise matris tersinir değildir (tekil).
Simetrik	$A = A^T$; kovaryans matrisleri simetriktir.

1.8 Özdeğerler ve Özvektörler

Bir kare matris A için, yönü değişmeyip yalnızca ölçeklenen özel vektörlere *özvektör*, ölçekleme çarpanına *özdeğer* denir:

$$Av = \lambda v, \quad v \neq 0.$$

Burada v özvektör, λ özdeğerdir. Özdeğerler, PCA'nın (Kısım 5) kalbindedir: bir veri kümesinin kovaryans matrisinin özvektörleri, verinin en çok yayıldığı yönleri (temel bileşenleri) verir; özdeğerler o yönlerdeki varyansı ölçer.

```

1 A = np.array([[2.0, 0.0],
2               [0.0, 3.0]])

```

```

3 eigenvalues, eigenvectors = np.linalg.eig(A)
4 # özdeğerler: [2., 3.]
5 # özvektörler: sütunlar özvektörlerdir

```

Bilgi

Neden matris biçimi? Bir döngüyle 100 örnek için tahmin hesaplamak yerine tek bir $X @ w$ çarpımı kullanmak hem çok daha hızlıdır (BLAS kütüphaneleri, GPU) hem de matematiği okunaklı kılar. Makine öğrenmesi kodunu “vektörleştirmek” (döngüleri matris işlemleriyle değiştirmek) temel bir beceridir.

2. Kalkülüs ve Optimizasyon

Makine öğrenmesinde “öğrenmek”, bir *kayıp fonksiyonunu* (modelin ne kadar yanlış olduğunu ölçen fonksiyon) minimize etmektir. Bunu yapmak için kalkülüs kullanırız: türev, fonksiyonun hangi yönde arttığını söyler; ters yöne giderek kaybı azaltırız. Bu, *gradient descent* (gradyan inişi) algoritmasının özüdür.

2.1 Türev: Değişim Oranı

Tek değişkenli bir $f(x)$ fonksiyonunun türevi $f'(x)$, fonksiyonun x noktasındaki anlık değişim oranıdır (eğim):

$$f'(x) = \frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Bir minimum (veya maksimum) noktada eğim sıfırdır: $f'(x) = 0$. Optimizasyonun temel fikri budur — kaybın türevinin sıfır olduğu parametreleri ararız.

$f(x)$	$f'(x)$	Kural
x^n	nx^{n-1}	Kuvvet kuralı
e^x	e^x	Üstel
$\ln x$	$1/x$	Logaritma
$f(g(x))$	$f'(g(x)) g'(x)$	Zincir kuralı (backprop'un temeli)

2.2 Kısmi Türev ve Gradyan

Makine öğrenmesinde kayıp, *birçok* parametreye bağlıdır: $L(w_1, w_2, \dots, w_n)$. Bir parametreye göre *kısmi türev* $\frac{\partial L}{\partial w_i}$, diğerlerini sabit tutup yalnızca w_i 'yi değiştirdiğimizde kaybın nasıl değiştiğini söyler. Tüm kısmi türevleri bir vektörde toplarsak *gradyanı* elde ederiz:

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right]^\top.$$

Gradyan, kaybın *en hızlı arttığı* yönü gösteren bir vektördür. Dolayısıyla kaybı azaltmak için gradyanın *tersi* yönde ilerleriz.

2.3 Zincir Kuralı: Backprop'un Kalbi

Zincir kuralı, *bileşke fonksiyonların* türevini verir ve derin öğrenmedeki geri yayılımın (backpropagation) temelidir. İç içe fonksiyonlarda türevler *çarpılarak* dışarıdan içeriye taşınır:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x).$$

Türetme

Örnek: $f(x) = (3x + 1)^2$ fonksiyonunun türevi. Dış fonksiyon $(\cdot)^2$, iç fonksiyon $3x + 1$. Zincir kuralı:

$$f'(x) = \underbrace{2(3x + 1)}_{\text{dışın türevi}} \cdot \underbrace{3}_{\text{için türevi}} = 6(3x + 1).$$

Daha uzun bir zincir, örneğin sigmoid içeren bir kayıp $L = (\sigma(wx))^2$, aynı mantıkla katman katman çarpılır: $\frac{dL}{dw} = 2\sigma(wx) \cdot \sigma'(wx) \cdot x$. Sinir ağları tam olarak bu “çarpım zincirini” otomatikleştirir.

2.4 Kısmi Türev ve Gradyanın Somut Hesabı

Kısmi türev kavramını sayısal bir örnekle pekiştirelim. Bir kaybın gradyanını *elle* hesaplamak, gradient descent'in ne yaptığını netleştirir.

Türetme

İki parametrelili kayıp: $L(w_1, w_2) = (w_1 - 2)^2 + (w_2 + 1)^2$. Bu, minimumu $(2, -1)$ 'de olan bir çanaktır. Kısmi türevler:

$$\frac{\partial L}{\partial w_1} = 2(w_1 - 2), \quad \frac{\partial L}{\partial w_2} = 2(w_2 + 1).$$

$(w_1, w_2) = (0, 0)$ noktasında gradyan:

$$\nabla L = [2(0 - 2), 2(0 + 1)]^\top = [-4, 2]^\top.$$

$\eta = 0.1$ ile bir gradient descent adımı:

$$\mathbf{w} \leftarrow [0, 0]^\top - 0.1 \cdot [-4, 2]^\top = [0.4, -0.2]^\top.$$

Görüldüğü gibi w_1 minimuma (2) doğru arttı, w_2 minimuma (-1) doğru azaldı — tam istediğimiz yön.

```
1 import numpy as np
2
3 # L(w) = (w1 - 2)^2 + (w2 + 1)^2 kaybının gradyanı
4 def grad_L(w):
5     return np.array([2 * (w[0] - 2), 2 * (w[1] + 1)])
6
7 w = np.array([0.0, 0.0]) # başlangıç noktası
8 lr = 0.1
9 for step in range(30):
10     w = w - lr * grad_L(w) # gradyanın tersine adım
11 print(w) # [2., -1.]'e yaklaşır (minimum)
```

2.5 Gradient Descent (Gradyan İnişi)

Gradient descent, kaybı minimize eden temel algoritmadır. Fikir basittir: parametreleri, gradyanın ters yönünde küçük adımlarla güncelle. Güncelleme kuralı:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w}),$$

burada $\eta > 0$ *öğrenme oranıdır* (learning rate) — adım büyüklüğünü kontrol eder. Bunu, sisli bir dağda aşağı inmeye benzet: her adımda ayağının altındaki en dik iniş yönünü bul ve o yöne bir adım at.

Türetme

$f(w) = w^2$ fonksiyonunu ele alalım; minimumu $w = 0$ 'dır. Türevi $f'(w) = 2w$. $w_0 = 5$ ve $\eta = 0.1$ ile başlayalım:

$$w_1 = 5 - 0.1 \cdot (2 \cdot 5) = 5 - 1 = 4, \quad w_2 = 4 - 0.1 \cdot 8 = 3.2, \quad w_3 = 3.2 - 0.1 \cdot 6.4 = 2.56, \dots$$

Her adımda w , minimuma (0) biraz daha yaklaşır. Adımlar minimuma yaklaştıkça (gradyan küçüldükçe) küçülür.

```

1 import numpy as np
2
3 def gradient_descent(grad_fn, start, learning_rate=0.1, n_iters=50):
4     """Genel amaçlı gradyan inişi.
5     grad_fn: bir noktadaki gradyanı döndüren fonksiyon.
6     """
7     w = start
8     history = [w]
9     for _ in range(n_iters):
10         grad = grad_fn(w)          # gradyanı hesapla
11         w = w - learning_rate * grad  # gradyanın tersine adım at
12         history.append(w)
13     return w, history
14
15 # f(w) = w^2'yi minimize et; gradyanı f'(w) = 2w
16 grad = lambda w: 2 * w
17 w_min, hist = gradient_descent(grad, start=5.0)
18 print(w_min)                      # 0.0'a yaklaşıyor

```

Dikkat

Öğrenme oranı η kritiktir. Çok küçükse öğrenme çok yavaş olur (binlerce adım). Çok büyükse adımlar minimumu aşar ve algoritma *ıraksar* (loss patlar, NaN olur). Pratikte η genelde 0.001–0.1 arasında denenir ve kayıp eğrisi izlenerek ayarlanır.

2.6 Gradient Descent Türleri

Tür	Her adımda kaç örnek?	Özellik
Batch GD	Tüm veri (m)	Kararlı ama yavaş, çok bellek.
Stochastic (SGD)	Tek örnek (1)	Hızlı, gürültülü, kaçış yapabilir.
Mini-batch	Küçük grup (32–256)	En yaygın; denge; GPU dostu.

2.7 Konvekslik: Neden Bazı Problemler Kolaydır?

Bir fonksiyon *konveksse* (dışbükey — çanak şeklinde) tek bir global minimumu vardır ve gradient descent oraya ulaşacağı garanti edilir. Lineer ve lojistik regresyonun kayıpları konvektir — bu yüzden “kolay” problemlerdir. Sinir ağlarının kaybı ise konveks *değildir*: birçok yerel minimum ve eyer noktası vardır; yine de gradient descent pratikte yeterince iyi çözümler bulur.

Bilgi

Otomatik türev (autograd): Modern kütüphaneler (PyTorch, TensorFlow, JAX) gradyanları elle hesaplamana gerek bırakmaz; hesaplama çizgesini takip edip zincir kuralını otomatik uygular. Yine de bu rehberde gradyanları elle türeteceğiz — çünkü “sihrin” altında ne olduğunu anlamak, hata ayıklamak ve yeni modeller tasarlamak için şarttır.

3. Olasılık ve İstatistik

Makine öğrenmesi belirsizlikle uğraşır: veri gürültülüdür, tahminler asla kesin değildir. Olasılık teorisi, bu belirsizliği matematiksel olarak modeller. Birçok ML algoritması (Naive Bayes, lojistik regresyon, tüm üretici modeller) doğrudan olasılık üzerine kuruludur.

3.1 Temel Kavramlar

Bir *rastgele değişken* X , sonucu belirsiz bir niceliktir. Olasılık dağılımı, X 'in hangi değerleri hangi olasılıkla aldığı tarif eder. Ayrık değişkenler için *olasılık kütle fonksiyonu* (PMF) $P(X = x)$, sürekli için *olasılık yoğunluk fonksiyonu* (PDF) $p(x)$ kullanılır. Temel büyüklükler:

$$\mathbb{E}[X] = \sum_x x P(X = x) \quad (\text{beklenen değer, ortalama}), \quad \text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Beklenen değer dağılımın “merkezini” (ortalama), varyans ise “yayılımını” (dağımlık) ölçer. Standart sapma $\sigma = \sqrt{\text{Var}(X)}$ 'dir.

Türetme

Örnek — adil zar: $X \in \{1, \dots, 6\}$, her yüz olasılığı $\frac{1}{6}$. Beklenen değer:

$$\mathbb{E}[X] = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = \frac{21}{6} = 3.5.$$

Varyans $\mathbb{E}[X^2] - (\mathbb{E}[X])^2$ ile: $\mathbb{E}[X^2] = \frac{1}{6}(1 + 4 + 9 + 16 + 25 + 36) = \frac{91}{6} \approx 15.17$, dolayısıyla $\text{Var}(X) = 15.17 - 3.5^2 = 2.92$.

3.2 Önemli Olasılık Dağılımları

ML'de sürekli karşılaşılan birkaç dağılımı tanımak, hangi modelin hangi varsayıma dayandığını anlamak için gereklidir.

Dağılım	Kütle/Yoğunluk	Ne modeller?
Bernoulli	$p^y(1-p)^{1-y}$	Tek ikili sonuç (0/1). Lojistik regresyon.
Binom	$\binom{n}{k}p^k(1-p)^{n-k}$	n denemede başarı sayısı.
Kategorik	$\prod_c p_c^{[y=c]}$	Çok sınıflı etiket. Softmax çıktısı.
Gaussian	$\frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	Sürekli, gürültü. En küçük kareler.

Normal (Gaussian) dağılım özellikle önemlidir: iki parametresi vardır — ortalama μ (merkez) ve standart sapma σ (genişlik). Doğadaki birçok gürültü normal dağıldığından, birazdan göreceğimiz gibi *en küçük kareler kaybı* bu varsayımdan doğar.

```
1 import numpy as np
```

```
2
```

```

3 rng = np.random.default_rng(42)           # tekrarlanabilirlik için tohum
4
5 bernoulli = rng.binomial(1, 0.3, size=1000) # p=0.3 ile 0/1 örnekler
6 gaussian = rng.normal(loc=0.0, scale=1.0, size=1000) # mu=0, sigma=1
7
8 print(bernoulli.mean())                     # ~0.3 (ampirik p)
9 print(gaussian.mean(), gaussian.std())      # ~0.0, ~1.0

```

3.3 Bağımsızlık ve Olasılık Zincir Kuralı

İki olay *bağımsızsa*, birlikte olma olasılıkları çarpımdır: $P(A \cap B) = P(A)P(B)$. Bu basit kural, Naive Bayes’in “saf” varsayımının (özellikler sınıf verildiğinde bağımsız) ve MLE’de olabilirliğin neden bir *çarpım* olduğunun temelidir. Genel olarak (bağımlı olsalar bile) zincir kuralı geçerlidir: $P(A \cap B) = P(A)P(B | A)$.

3.4 Koşullu Olasılık ve Bayes Teoremi

Koşullu olasılık $P(A | B)$, “ B gerçekleştiği bilindiğinde A ’nın olasılığı”dır. Buradan makine öğrenmesinin en önemli formüllerinden biri olan *Bayes teoremi* çıkar:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}.$$

Sınıflandırma diliyle: bir örneğin özellikleri $\mathbf{x}$ verildiğinde, sınıf y olma olasılığı (*posterior*):

$$\underbrace{P(y | \mathbf{x})}_{\text{sonsal}} = \frac{\overbrace{P(\mathbf{x} | y)}^{\text{olabilirlik}} \overbrace{P(y)}^{\text{önsel}}}{\underbrace{P(\mathbf{x})}_{\text{kanıt}}}.$$

Bu formül, Naive Bayes sınıflandırıcısının (Kısım 4) doğrudan temelidir: her sınıf için sağ tarafı hesaplar ve en yüksek olasılıklı sınıfı seçeriz.

Türetme

Örnek — tıbbi test: Bir hastalık nüfusun %1’inde var ($P(H) = 0.01$). Test hastalığı %99 yakalıyor ($P(+ | H) = 0.99$) ama sağlıklıda da %5 yanlış alarm veriyor ($P(+ | \bar{H}) = 0.05$). Testin pozitif çıktı; gerçekten hasta olma olasılığın nedir?

$$P(H | +) = \frac{P(+ | H)P(H)}{P(+ | H)P(H) + P(+ | \bar{H})P(\bar{H})} = \frac{0.99 \cdot 0.01}{0.99 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.167.$$

Yani pozitif teste rağmen gerçekten hasta olma olasılığın yalnızca %17! Nadir olayların (düşük önsel) sezgiye aykırı sonuçları buradan gelir.

3.5 Maksimum Olabilirlik Tahmini (MLE)

Maksimum olabilirlik (Maximum Likelihood Estimation), bir modelin parametrelerini veriden tahmin etmenin temel yöntemidir ve birçok kayıp fonksiyonunun *nereden geldiğini* açıklar. Fikir: gözlemlediğimiz veriyi *en olası* kılan parametreleri seç.

Bağımsız gözlemler $x_1, \dots, x_m$ için olabilirlik, çarpımdır:

$$\mathcal{L}(\theta) = \prod_{i=1}^m p(x_i | \theta).$$

Çarpımlarla çalışmak zor olduğundan (ve sayısal olarak taşabildiğinden) *log-olabilirliği* maksimize ederiz (log tekdüze arttığından maksimum noktası aynıdır):

$$\ell(\theta) = \log \mathcal{L}(\theta) = \sum_{i=1}^m \log p(x_i | \theta).$$

Türetme

Basit örnek — hileli para: Bir parayı 10 kez attık ve 7 kez yazı geldi. Yazı olasılığı p 'yi MLE ile tahmin edelim. Olabilirlik (Bernoulli): $\mathcal{L}(p) = p^7(1-p)^3$. Log-olabilirlik:

$$\ell(p) = 7 \ln p + 3 \ln(1-p).$$

Maksimum için türevi sıfırlarız:

$$\ell'(p) = \frac{7}{p} - \frac{3}{1-p} = 0 \implies 7(1-p) = 3p \implies p = \frac{7}{10}.$$

Yani MLE tahmini $\hat{p} = 0.7$ — tam da ampirik frekans (yazı sayısı / toplam). Sezgimizle örtüşür; MLE'nin gücü, sezgisel olmayan modellerde de aynı ilkeyi uygulayabilmesidir.

Türetme

MLE $\Rightarrow$ En Küçük Kareler: Regresyonda hataların normal dağıldığını, $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$ ve $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, varsayalım. Bir örneğin olabilirliği:

$$p(y_i | \mathbf{x}_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{w}^\top \mathbf{x}_i)^2}{2\sigma^2}\right).$$

Log-olabilirlik toplandığında, $\mathbf{w}$ 'ye bağlı olmayan sabitler atılınca:

$$\ell(\mathbf{w}) = -\frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \text{sabit}.$$

ℓ 'i maksimize etmek, $\sum (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$ toplamını *minimize* etmekle aynıdır — yani **ortalama kare hatası!** Demek ki en küçük kareler, Gauss gürültü varsayımı altında MLE'dir.

Bilgi

Bu türetme çok önemli bir dersi öğretir: makine öğrenmesindeki kayıp fonksiyonları keyfi değildir. Ortalama kare hatası (MSE) normal dağılımdan, çapraz entropi (cross-entropy) Bernoulli/kategorik dağılımdan MLE yoluyla doğar. Kaybı seçmek, aslında verinin dağılımı hakkında bir varsayım yapmaktır.

4. Bilgi Teorisi

Bilgi teorisi, “belirsizliği” ve “bilgiyi” niceliksel ölçer. Sınıflandırma kayıplarının (cross-entropy) ve karar ağaçlarının bölme kriterlerinin (information gain) temelini oluşturur.

4.1 Entropi

Entropi, bir olasılık dağılımının *belirsizliğini* ölçer. Bir olayın taşıdığı “şaşırtıcılık” $-\log p$ ile tanımlanır (nadir olaylar daha şaşırtıcı). Entropi, bu şaşırtıcılığın beklenen değeridir:

$$H(P) = -\sum_i p_i \log p_i.$$

Dağılım ne kadar düzgünse (belirsizse) entropi o kadar yüksektir. Adil bir paranın entropisi maksimumdur ($\log 2 = 1$ bit); hep yazı gelen hileli bir paranın entropisi sıfırdır (belirsizlik yok).

```
1 import numpy as np
```

```

2
3 def entropy(p):
4     """Bir olasılık dağılımının Shannon entropisi (bit cinsinden)."""
5     p = np.asarray(p)
6     p = p[p > 0] # log(0)'dan kaçın
7     return -np.sum(p * np.log2(p))
8
9 print(entropy([0.5, 0.5])) # 1.0 bit (maksimum belirsizlik)
10 print(entropy([0.9, 0.1])) # 0.469 bit
11 print(entropy([1.0, 0.0])) # 0.0 bit (belirsizlik yok)

```

4.2 Çapraz Entropi ve KL İraksaması

Çapraz entropi, gerçek dağılım P yerine tahmin edilen dağılım Q kullanıldığında gereken ortalama şaşırtıcılıktır:

$$H(P, Q) = - \sum_i p_i \log q_i.$$

Sınıflandırmada P gerçek etiket (one-hot), Q modelin tahmin ettiği olasılıklardır. Çapraz entropiyi minimize etmek, modelin tahminlerini gerçek etiketlere yaklaştırır — bu yüzden sınıflandırmanın standart kaybıdır.

İkili sınıflandırmada ($y \in \{0, 1\}$) çapraz entropi, tek bir örnek için şu tanıdık biçimi alır (*binary cross-entropy*):

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})],$$

burada $\hat{y}$ modelin “sınıf 1” olma tahminidir. Kısım 3’te bunun Bernoulli MLE’sinden doğduğunu göreceğiz.

Türetme

Örnek — çapraz entropi hesabı: Gerçek etiket 3 sınıflı, one-hot $P = (0, 1, 0)$ (yani doğru sınıf ikinci). Model iki farklı tahmin üretsın:

- İyi model $Q = (0.1, 0.8, 0.1)$: $L = -\ln(0.8) \approx 0.22$.
- Kötü model $Q = (0.4, 0.3, 0.3)$: $L = -\ln(0.3) \approx 1.20$.

Yalnızca doğru sınıfa atanan olasılığın logaritması sayılır (diğer terimler $p_i = 0$ ile çarpılıp yok olur). Model doğru sınıfa ne kadar yüksek olasılık verirse kayıp o kadar küçük olur — doğruya 1 verirse kayıp 0.

```

1 import numpy as np
2
3 def cross_entropy(p_true, q_pred):
4     """One-hot gerçek dağılım ile tahmin arası çapraz entropi (doğal log)."""
5     q_pred = np.clip(q_pred, 1e-12, 1.0) # log(0)'ı önle
6     return -np.sum(p_true * np.log(q_pred))
7
8 p = np.array([0, 1, 0]) # gerçek sınıf: ikinci
9 print(cross_entropy(p, [0.1, 0.8, 0.1])) # ~0.22 (iyi tahmin)
10 print(cross_entropy(p, [0.4, 0.3, 0.3])) # ~1.20 (kötü tahmin)

```

Kullback-Leibler (KL) ıraksaması, iki dağılım arasındaki “uzaklığı” ölçer:

$$D_{\text{KL}}(P \parallel Q) = \sum_i p_i \log \frac{p_i}{q_i} = H(P, Q) - H(P).$$

KL her zaman ≥ 0 ’dır ve yalnızca $P = Q$ iken sıfırdır. Gerçek dağılım P sabit olduğunda $H(P)$ de sabittir; dolayısıyla çapraz entropiyi minimize etmek, KL ıraksamasını minimize etmekle aynıdır.

Kavram	Ölçtüğü	ML’de kullanımı
Entropi	Bir dağılımın belirsizliği	Karar ağacı bölme (information gain)
Çapraz entropi	Q ile P ’yi kodlama maliyeti	Sınıflandırma kaybı
KL ıraksaması	İki dağılım arası fark	VAE, dağılım eşleştirme

Bağlantı

Kısım 3’te lojistik regresyonun kaybını türettiğimizde, Bernoulli dağılımı için MLE’nin tam olarak *ikili çapraz entropiye* eşit olduğunu göreceğiz. Böylece olasılık (MLE), bilgi teorisi (cross-entropy) ve optimizasyon (gradient descent) tek bir noktada birleşecek. Bu üç temelin nasıl kaynaştığını görmek, makine öğrenmesini gerçekten “anlamak” demektir.

KISIM 2

Öğrenme Teorisi

5. Öğrenme Problemi ve Kayıp Fonksiyonları

Makine öğrenmesini kurmadan önce “öğrenmenin” ne demek olduğunu matematiksel olarak tanımlamalıyız. Denetimli öğrenmede elimizde girdi-çıkı çiftleri $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ vardır ve amacımız, yeni bir $\mathbf{x}$ için y 'yi tahmin eden bir $f_{\theta}(\mathbf{x})$ fonksiyonu (model) bulmaktır.

5.1 Temel Bileşenler

Bileşen	Tanım
Model f_{θ}	Girdiden çıktıya eşleyen, θ parametrelili fonksiyon.
Kayıp L	Tek bir tahminin ne kadar yanlış olduğu.
Maliyet J	Tüm veri üzerinde ortalama kayıp: $J(\theta) = \frac{1}{m} \sum_i L(f_{\theta}(\mathbf{x}_i), y_i)$.
Optimizasyon	J 'yi minimize eden θ 'yı bulma (öğrenme budur).

5.2 Yaygın Kayıp Fonksiyonları

Kayıp fonksiyonu, “iyi tahmin” tanımını belirler ve probleme göre seçilir.

Kayıp	Formül	Kullanım
Kare hata (MSE)	$(y - \hat{y})^2$	Regresyon
Mutlak hata (MAE)	$ y - \hat{y} $	Regresyon (aykırıya dayanıklı)
İkili çapraz entropi	$-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$	İkili sınıflandırma
Hinge	$\max(0, 1 - y\hat{y})$	SVM

5.3 Ampirik Risk Minimizasyonu

Gerçekte veriyi üreten dağılımı bilmeyiz; elimizde yalnızca örneklem vardır. Bu yüzden “gerçek riski” (tüm dağılım üzerinde beklenen kayıp) doğrudan minimize edemeyiz; onun yerine *ampirik riski* (eğitim verisi üzerindeki ortalama kayıp) minimize ederiz:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m L(f_{\theta}(\mathbf{x}_i), y_i).$$

Buradaki tehlike şudur: eğitim verisinde mükemmel olmak, *yeni* veride iyi olmayı garanti etmez. Bu, aşırı öğrenme problemidir.

6. Aşırı Öğrenme, Bias-Variance ve Düzenleştirme

6.1 Aşırı ve Eksik Öğrenme

Bir modelin başarısı, eğitim verisini ezberlemesiyle değil, *yeni* veriye *genelleme* yapmasıyla ölçülür. İki başarısızlık modu vardır:

Durum	Belirti	Neden / çözüm
Eksik öğrenme	Eğitim VE test kötü	Model çok basit; daha güçlü model / özellik.
İyi denge	Her ikisi de iyi	İstenen nokta.
Aşırı öğrenme	Eğitim iyi, test kötü	Model çok karmaşık; düzenleştirme / daha çok veri.

6.2 Bias-Variance Ayrışması

Bir modelin beklenen test hatası matematiksel olarak üç parçaya ayrılır:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}[\hat{f}]^2}_{\text{yanlılık}} + \underbrace{\text{Var}[\hat{f}]}_{\text{varyans}} + \underbrace{\sigma^2}_{\text{indirgenemez gürültü}}.$$

Bias (yanlılık): modelin sistematik hatası — çok basit bir model gerçeği kaçırır (eksik öğrenme).
Variance (varyans): modelin eğitim verisindeki küçük değişimlere aşırı duyarlılığı — çok karmaşık model gürültüyü öğrenir (aşırı öğrenme). İkisini aynı anda düşürmek genelde mümkün değildir; bu *bias-variance ödünleşmesidir*.

Bilgi

Model karmaşıklığı arttıkça bias düşer ama variance artar. Toplam hata önce düşer, bir minimum-dan sonra tekrar yükselir. Amaç, bu *tath noktayı* bulmaktır — ne fazla basit ne fazla karmaşık. Düzenleştirme ve çapraz doğrulama bunun için araçlardır.

6.3 Düzenleştirme (Regularization)

Düzenleştirme, aşırı öğrenmeyi önlemek için maliyet fonksiyonuna model karmaşıklığını cezalandıran bir terim ekler. Böylece model, veriye uymakla *basit kalmak* arasında denge kurar:

$$J(\mathbf{w}) = \underbrace{\frac{1}{m} \sum_{i=1}^m L(f_{\mathbf{w}}(\mathbf{x}_i), y_i)}_{\text{veriye uyum}} + \lambda \underbrace{R(\mathbf{w})}_{\text{karmaşıklık cezası}}.$$

Burada $\lambda \geq 0$ ceza gücünü ayarlar. İki temel ceza:

$$\text{Ridge (L2): } R(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_j w_j^2, \quad \text{Lasso (L1): } R(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_j |w_j|.$$

L2 ağırlıkları küçültür ama sıfırlamaz; L1 bazı ağırlıkları *tam sıfır* yapar, böylece otomatik özellik seçimi sağlar. Bu farkın nedenini Kısım 3'te göreceğiz.

Dikkat

λ bir hiperparametredir — veriden öğrenilmez, çapraz doğrulamayla seçilir. λ çok büyükse model aşırı basitleşir (eksik öğrenme, yüksek bias); çok küçükse düzenleştirme etkisiz kalır (aşırı öğrenme, yüksek variance). Doğru λ 'yı bir doğrulama kümesinde aranmalıdır.

KISIM 3

Regresyon

7. Lineer Regresyon

Lineer regresyon, makine öğrenmesinin “merhaba dünya”sidir: basit ama üzerine çok şey inşa edilen temel bir modeldir. Çıktının, girdilerin *ağırlıklı toplamı* olduğunu varsayar.

7.1 Model

n özellikli bir örnek $\mathbf{x} = (x_1, \dots, x_n)$ için tahmin:

$$\hat{y} = w_0 + w_1x_1 + \dots + w_nx_n = \mathbf{w}^\top \mathbf{x},$$

burada w_0 bias (kesişim) terimidir; $\mathbf{x}$ 'e sabit bir 1 eklenerek (bias trick) matris çarpımına dahil edilir. Tüm veri için: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$, $\mathbf{X} \in \mathbb{R}^{m \times (n+1)}$.

7.2 Maliyet Fonksiyonu

Kısım 1'de MLE'den türettiğimiz gibi, kayıp ortalama kare hatasıdır:

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 = \frac{1}{2m} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2.$$

$\frac{1}{2}$ çarpanı yalnızca türevi sadeleştirmek içindir (sonuçları değiştirmez). Bu J konvektir, dolayısıyla tek bir global minimumu vardır.

7.3 Çözüm 1: Normal Denklem (Kapalı Form)

Konveks J 'nin minimumunu gradyanını sıfıra eşitleyerek *analitik olarak* bulabiliriz.

Türetme

Maliyeti matris biçiminde açalım:

$$J(\mathbf{w}) = \frac{1}{2m}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}).$$

$\mathbf{w}$ 'ye göre gradyan (matris kalkülüsü kuralları ile):

$$\nabla_{\mathbf{w}} J = \frac{1}{m} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}).$$

Sıfıra eşitleyip çözersek *normal denklemi* elde ederiz:

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y} \implies \boxed{\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}}.$$

```

1 import numpy as np
2
3 class LinearRegressionClosedForm:
4     def fit(self, X, y):
5         # bir'lerden oluşan bias sütunu ekle -> X (m, n+1) olur
6         X_b = np.c_[np.ones((X.shape[0], 1)), X]
7         # Normal denklem: w = (X^T X)^{-1} X^T y
8         self.w = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
9         return self
10
11     def predict(self, X):
12         X_b = np.c_[np.ones((X.shape[0], 1)), X]
13         return X_b @ self.w
14
15 # Örnek
16 X = np.array([[1.0], [2.0], [3.0], [4.0]])
17 y = np.array([2.0, 4.0, 6.0, 8.0]) # tam olarak y = 2x
18 model = LinearRegressionClosedForm().fit(X, y)
19 print(model.w) # ~[0, 2] (bias 0, eğim 2)

```

Dikkat

Normal denklem $(\mathbf{X}^\top \mathbf{X})^{-1}$ matris tersi gerektirir; bu $\mathcal{O}(n^3)$ 'tür ve özellik sayısı n çok büyükse (binlerce) pahalıdır. Ayrıca $\mathbf{X}^\top \mathbf{X}$ tekil (tersinemez) olabilir. Bu durumlarda gradient descent tercih edilir.

7.4 Çözüm 2: Gradient Descent

Büyük veri için normal denklem yerine gradyan inişini kullanırız. Yukarıda türettiğimiz gradyanı doğrudan kullanan güncelleme:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{m} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}).$$

```

1 class LinearRegressionGD:
2     def __init__(self, lr=0.01, n_iters=1000):
3         self.lr = lr
4         self.n_iters = n_iters
5
6     def fit(self, X, y):

```

```

7     m, n = X.shape
8     self.w = np.zeros(n)
9     self.b = 0.0
10    for _ in range(self.n_iters):
11        y_pred = X @ self.w + self.b           # tahminler (m,)
12        error = y_pred - y                     # artıklar (m,)
13        # MSE'nin ağırlıklara ve bias'a göre gradyanları
14        dw = (1 / m) * (X.T @ error)           # (n,)
15        db = (1 / m) * np.sum(error)           # skaler
16        self.w -= self.lr * dw                 # gradyan adımı
17        self.b -= self.lr * db
18    return self
19
20    def predict(self, X):
21        return X @ self.w + self.b

```

7.5 scikit-learn ile

```

1  from sklearn.linear_model import LinearRegression
2  from sklearn.metrics import mean_squared_error, r2_score
3
4  model = LinearRegression()
5  model.fit(X_train, y_train)
6  y_pred = model.predict(X_test)
7
8  print(model.coef_, model.intercept_)         # öğrenilen ağırlıklar ve bias
9  print(r2_score(y_test, y_pred))             # uyum iyiliği

```

8. Ridge, Lasso ve Lojistik Regresyon

8.1 Ridge Regresyonu (L2)

Ridge, maliyete $L2$ cezası ekler. İlginç olan, kapalı formunun hâlâ mevcut olması ve tekillik problemini de çözmesidir:

$$J(\mathbf{w}) = \frac{1}{2m} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \implies \mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$

$\lambda \mathbf{I}$ terimi köşegeni güçlendirir, matrisi her zaman tersinir kılar — bu yüzden Ridge sayısal olarak da karardır.

8.2 Neden Lasso Sıfırlar, Ridge Sıfırlamaz?

Bu, sıkça sorulan ve derin bir sorudur. Geometrik sezgi: düzenleştirme, ağırlıkları bir “bütçe bölgesi” içinde kalmaya zorlar. $L2$ bölgesi bir *daire* (yuvarlak), $L1$ bölgesi bir *karedir* (köşeli). Maliyet konturları bu bölgeye ilk değdiği noktada çözüm bulunur. $L1$ 'in köşeleri eksenler üzerindedir — kontur genelde bir köşeye değer ve o eksenindeki ağırlık tam sıfır olur. $L2$ 'nin yuvarlak sınırında böyle köşe yoktur, bu yüzden sıfırlamaz yalnızca küçültür.

	Ridge (L2)	Lasso (L1)
Ceza	$\sum w_j^2$	$\sum w_j $
Ağırlıklara etki	Küçültür (sıfırlamaz)	Bazılarını sıfırlar
Özellik seçimi	Hayır	Evet (seyrek çözüm)
Kapalı form	Var	Yok (koordinat inisi)

```

1 from sklearn.linear_model import Ridge, Lasso, ElasticNet
2
3 ridge = Ridge(alpha=1.0).fit(X_train, y_train)
4 lasso = Lasso(alpha=0.1).fit(X_train, y_train)      # bazı coef_ 0 olur
5 elastic = ElasticNet(alpha=0.1, l1_ratio=0.5)      # L1 + L2 birleşik
6 print(lasso.coef_)                                # sıfırları gözlemle

```

8.3 Lojistik Regresyon: Sınıflandırmaya Köprü

Adına “regresyon” dense de lojistik regresyon bir *sınıflandırma* algoritmasıdır. Lineer bir skoru, *sigmoid* fonksiyonuyla $[0, 1]$ aralığına — bir olasılığa — sıkıştırır:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{y} = P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}).$$

Sigmoid’in güzel bir özelliği türevidir: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ — bu, gradyanı sadeleştirir.

8.4 Kayıp: İkili Çapraz Entropi (MLE’den)

Türetme

Etiket $y \in \{0, 1\}$ Bernoulli dağılır: $P(y \mid \mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y}$. Log-olabilirlik, tek örnek için:

$$\log P(y \mid \mathbf{x}) = y \log \hat{y} + (1 - y) \log(1 - \hat{y}).$$

Bunun *negatifini* minimize etmek, tüm veri için maliyeti verir — **ikili çapraz entropi**:

$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)].$$

Gradyanı türetince (sigmoid türevi sayesinde) çarpıcı biçimde sadeleşir — lineer regresyonunkiyle *aynı* biçimi alır:

$$\nabla_{\mathbf{w}} J = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}).$$

```

1 import numpy as np
2
3 class LogisticRegressionScratch:
4     def __init__(self, lr=0.1, n_iters=1000):
5         self.lr, self.n_iters = lr, n_iters
6
7     @staticmethod
8     def _sigmoid(z):
9         return 1.0 / (1.0 + np.exp(-z))
10
11     def fit(self, X, y):

```

```

12     m, n = X.shape
13     self.w = np.zeros(n)
14     self.b = 0.0
15     for _ in range(self.n_iters):
16         z = X @ self.w + self.b
17         y_hat = self._sigmoid(z) # tahmin edilen olasılıklar
18         # Lineer regresyonla aynı gradyan biçimi (MLE türetmesiyle)
19         dw = (1 / m) * (X.T @ (y_hat - y))
20         db = (1 / m) * np.sum(y_hat - y)
21         self.w -= self.lr * dw
22         self.b -= self.lr * db
23     return self
24
25     def predict_proba(self, X):
26         return self._sigmoid(X @ self.w + self.b)
27
28     def predict(self, X, threshold=0.5):
29         return (self.predict_proba(X) >= threshold).astype(int)

```

Bilgi

Lineer ve lojistik regresyonun gradyanının *aynı biçimde* ($\mathbf{X}^\top(\hat{\mathbf{y}} - \mathbf{y})$) çıkması tesadüf değildir: her ikisi de “genelleştirilmiş lineer modeller” ailesindendir. Bu ortak yapı, aynı optimizasyon kodunu farklı modellerde yeniden kullanmanı sağlar — ve derin öğrenmedeki katman gradyanlarının da temelini oluşturur.

KISIM 4

Sınıflandırma Algoritmaları

9. Naive Bayes ve K-En Yakın Komşu

9.1 Naive Bayes

Naive Bayes, Kısım 1'deki Bayes teoreminin doğrudan uygulamasıdır. Bir örneğin her sınıfa ait olma olasılığını hesaplar ve en yükseğini seçer:

$$\hat{y} = \arg \max_y P(y | \mathbf{x}) = \arg \max_y P(\mathbf{x} | y) P(y).$$

“Naive” (saf) olmasının nedeni, güçlü bir varsayımdır: özelliklerin, sınıf verildiğinde *birbirinden bağımsız* olduğunu kabul eder. Bu varsayım sayesinde ortak olasılık, tek tek olasılıkların çarpımına iner:

$$P(\mathbf{x} | y) = \prod_{j=1}^n P(x_j | y).$$

Varsayım gerçekçi olmasa da (özellikler genelde ilişkilidir), pratikte — özellikle metin sınıflandırmada (spam filtresi) — şaşırtıcı derecede iyi çalışır ve çok hızlıdır.

Türetme

Sayısal taşmayı önlemek için çarpım yerine *log* kullanılır (log, çarpımı toplama çevirir ve tekdüze olduğundan argmax'i değiştirmez):

$$\hat{y} = \arg \max_y \left[\log P(y) + \sum_{j=1}^n \log P(x_j | y) \right].$$

Sürekli özellikler için $P(x_j | y)$ genelde normal (Gaussian) dağılımla modellenir; her sınıf-özellik çifti için ortalama ve varyans eğitim verisinden tahmin edilir.

```
1 from sklearn.naive_bayes import GaussianNB, MultinomialNB
2
3 gnb = GaussianNB()                                # sürekli özellikler
4 gnb.fit(X_train, y_train)
5 print(gnb.predict(X_test))
6
7 # Metin (kelime sayıları) için MultinomialNB standarttır:
8 # mnb = MultinomialNB(alpha=1.0) # alpha = Laplace düzeltmesi
```


Dikkat

Sıfır olasılık problemi: Eğitimde hiç görülmemiş bir özellik-sınıf kombinasyonu olasılığı 0 yapar ve tüm çarpımı sıfırlar. Çözüm *Laplace düzeltmesidir* (smoothing): her sayıya küçük bir sabit (α) eklenir, böylece hiçbir olasılık tam sıfır olmaz.

9.2 K-En Yakın Komşu (KNN)

KNN, “benzer örnekler benzer etikete sahiptir” sezgisine dayanan, eğitimsiz (lazy) bir algoritmadır. Yeni bir örneği sınıflandırmak için: eğitim kümesindeki *en yakın k* komşuyu bul, çoğunluğun etiketini ver. “Yakınlık” genelde Öklid uzaklığıyla ölçülür:

$$d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_2 = \sqrt{\sum_{j=1}^n (x_j - x'_j)^2}.$$

KNN’nin “eğitimi” yoktur — tüm işi tahmin anında yapar (veriyi saklar, sorguda uzaklıkları hesaplar). Bu yüzden büyük veride tahmin yavaştır.

```

1 import numpy as np
2 from collections import Counter
3
4 class KNNClassifier:
5     def __init__(self, k=5):
6         self.k = k
7
8     def fit(self, X, y):
9         self.X_train = X          # tembel: veriyi sadece ezberle
10        self.y_train = y
11        return self
12
13    def predict(self, X):
14        preds = []
15        for x in X:
16            # her eğitim noktasına Öklid uzaklığı
17            distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))
18            k_idx = np.argsort(distances)[:self.k]      # en yakın k indeks
19            k_labels = self.y_train[k_idx]
20            preds.append(Counter(k_labels).most_common(1)[0][0]) # çoğunluk
21        return np.array(preds)

```

Bilgi

k seçimi bir ödünleşmedir: Küçük *k* (örneğin 1) gürültüye duyarlıdır (yüksek variance); büyük *k* sınırları aşırı yumuşatır (yüksek bias). KNN *ölçeğe çok duyarlıdır* — büyük değerli özellikler uzaklığa hâkim olur; bu yüzden KNN’den önce mutlaka özellikleri standartlaştır.

10. Destek Vektör Makineleri (SVM)

SVM, iki sınıfı ayıran en iyi *hiperdüzlemi* bulan güçlü bir sınıflandırıcıdır. “En iyi” derken: iki sınıf arasındaki boşluğu (marjı) *maksimize* eden düzlemi kasteder. Geniş marj, daha iyi genelleme demektir.

10.1 Maksimum Marj

Bir hiperdüzlem $\mathbf{w}^\top \mathbf{x} + b = 0$ ile tanımlanır. Sınıflar $y \in \{-1, +1\}$ olsun. Her örneğin düzleme uzaklığının en azını (marjı) maksimize etmek istiyoruz. Bu, şu kısıtlı optimizasyona dönüşür:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{öyle ki} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i.$$

Marj $\frac{2}{\|\mathbf{w}\|}$ olduğundan, onu maksimize etmek $\|\mathbf{w}\|$ 'yi minimize etmekle aynıdır. Kısıtı sağlayan, marj sınırında yatan örnekler *destek vektörleri* denir — çözümü yalnızca onlar belirler.

10.2 Yumuşak Marj (Soft Margin)

Gerçek veri genelde tam ayrılabilir değildir. *Yumuşak marj*, bir miktar ihlale izin verir; bunu ξ_i “gevşeklik” değişkenleri ve bir C parametresiyle dengeler:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i, \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

C ödünleşmeyi kontrol eder: büyük C ihlalleri sert cezalandırır (dar marj, aşırı öğrenme riski); küçük C daha geniş marja izin verir (daha çok düzenleştirme).

10.3 Kernel Hilesi (Kernel Trick)

Veri doğrusal ayrılabilir değilse, SVM'i *daha yüksek boyutlu* bir uzaya taşıyız; orada doğrusal ayrılabilir hale gelebilir. Kernel hilesi, bu dönüşümü *açıkça hesaplamadan*, yalnızca iç çarpımlar üzerinden yapar:

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}').$$

Kernel	$K(\mathbf{x}, \mathbf{x}')$	Kullanım
Lineer	$\mathbf{x}^\top \mathbf{x}'$	Doğrusal ayrılabilir, çok özellik
Polinom	$(\gamma \mathbf{x}^\top \mathbf{x}' + r)^d$	Polinom sınırlar
RBF (Gauss)	$\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	En yaygın; esnek sınırlar

```

1 from sklearn.svm import SVC
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4
5 # SVM ölçeğe duyarlıdır -> önce her zaman standartlaştır
6 model = make_pipeline(
7     StandardScaler(),
8     SVC(kernel='rbf', C=1.0, gamma='scale') # RBF çekirdeği
9 )
10 model.fit(X_train, y_train)
11 print(model.score(X_test, y_test))

```

11. Karar Ağaçları

Karar ağacı, veriyi bir dizi “evet/hayır” sorusuyla bölerek sınıflandıran, sezgisel ve yorumlanabilir bir modeldir. Her iç düğüm bir özellik üzerinde bir eşik testi, her yaprak bir tahmindir. İnsanın karar verme sürecine benzediği için “beyaz kutu” modeldir — neden o kararı verdiğini açıklayabilirsiniz.

11.1 Bölme Kriteri: Bilgi Kazancı ve Gini

Ağaç, her adımda veriyi *en saf* alt gruplara bölen özelliği/eşiği seçer. Saflık iki şekilde ölçülür. Kısım 1'deki entropiyi kullanan *bilgi kazancı*:

$$IG = H(\text{ebeveyn}) - \sum_k \frac{m_k}{m} H(\text{çocuk}_k),$$

veya hesaplaması daha ucuz olan *Gini safsızlığı*:

$$\text{Gini} = 1 - \sum_c p_c^2,$$

burada p_c , düğümdeki c sınıfının oranıdır. Her ikisi de düğüm “saf” (tek sınıf) olduğunda sıfırdır. Ağaç, bölme sonrası safsızlığı en çok azaltan seçimi yapar.

Türetme

Örnek: 10 örnekli bir düğümde 5 pozitif, 5 negatif olsun. $\text{Gini} = 1 - (0.5^2 + 0.5^2) = 0.5$ (maksimum karışıklık). Bir bölme, sol düğüme 4 pozitif 1 negatif, sağa 1 pozitif 4 negatif koyarsa: her çocuğun Gini'si $1 - (0.8^2 + 0.2^2) = 0.32$; ağırlıklı ortalama 0.32. Safsızlık $0.5 \rightarrow 0.32$ düştü — iyi bir bölme.

```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2
3 tree = DecisionTreeClassifier(
4     criterion='gini',          # veya 'entropy'
5     max_depth=4,              # aşırı öğrenmeyi önlemek için derinliği sınırla
6     min_samples_leaf=5)
7 tree.fit(X_train, y_train)
8 print(tree.feature_importances_) # en çok hangi özellikler önemli
```

Dikkat

Karar ağaçları *aşırı öğrenmeye çok eğilimlidir*: sınırlamazsan her örneği ezberleyen dev bir ağaç kurar (eğitimde %100, testte kötü). Bunu `max_depth`, `min_samples_leaf` gibi parametrelerle “budarsın” (pruning). Ama tek ağacın asıl gücü, aşağıdaki topluluk yöntemlerinde ortaya çıkar.

12. Topluluk Yöntemleri (Ensemble)

Topluluk yöntemleri, birçok “zayıf” modeli birleştirerek tek başına her birinden daha güçlü bir model oluşturur. “Kalabalığın bilgeliği” ilkesi: bağımsız hataları olan çok sayıda modelin ortalaması, bireysel hataları söndürür. İki ana strateji vardır: bagging ve boosting.

12.1 Bagging ve Random Forest

Bagging (Bootstrap Aggregating), eğitim verisinden rastgele (yerine koyarak) alt örneklemeler çeker, her birinde ayrı bir ağaç eğitir ve tahminleri oylar/ortalalar. Bu, variance'ı düşürür (aşırı öğrenmeyi azaltır).

Random Forest, bagging'e bir ek yapar: her bölmede yalnızca özelliklerin rastgele bir alt kümesini dener. Bu, ağaçları birbirinden *ilişkisizleştirir* — ağaçlar ne kadar farklı hata yaparsa, ortalamaları o kadar iyi olur.

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rf = RandomForestClassifier(
4     n_estimators=200,          # ağaç sayısı
5     max_depth=None,
```

```

6     max_features='sqrt',      # her bölme için rastgele özellik alt kümesi
7     random_state=42)
8 rf.fit(X_train, y_train)
9 print(rf.feature_importances_)

```

12.2 Boosting: Hataları Ardışık Düzeltmek

Boosting, bagging'in aksine ağaçları *paralel* değil *ardışık* kurar: her yeni model, önceki modellerin *hatalarına* odaklanır. Gradient Boosting bunu, her adımda önceki modelin *artıklarına* (residual) yeni bir ağaç uydurarak yapar — adeta kaybın gradyanına gradient descent uygular.

	Bagging (RF)	Boosting
Ağaçlar	Paralel, bağımsız	Ardışık, birbirine bağlı
Azalttığı	Variance	Bias
Aşırı öğrenme	Dayanıklı	Daha hassas (dikkatli ayar)
Hız	Paralleleleşir	Sıralı (yavaş)

```

1 from sklearn.ensemble import GradientBoostingClassifier
2 # from xgboost import XGBClassifier      # popüler, daha hızlı alternatif
3
4 gb = GradientBoostingClassifier(
5     n_estimators=100,
6     learning_rate=0.1,      # her ağacın katkısını küçültür
7     max_depth=3)           # boosting sığ "zayıf" ağaçlar kullanır
8 gb.fit(X_train, y_train)

```

İpucu

Tablo (tabular) verisinde **gradient boosting** (XGBoost, LightGBM, CatBoost) genelde en iyi performansı verir ve Kaggle yarışmalarının çoğunu kazanır. Random Forest ise daha az ayar gerektiren, sağlam bir ilk tercihtir. Derin öğrenme, tablo verisinde çoğu zaman bu ağaç tabanlı yöntemleri geçemez — asıl gücünü görüntü, ses ve metinde gösterir.

KISIM 5

Denetimsiz Öğrenme

13. K-Means Kümeleme

Denetimsiz öğrenmede etiket yoktur; amaç veride gizli yapı bulmaktır. K-Means, veriyi k kümeye ayırır — her kümeyi bir *merkez* (centroid) temsil eder ve her nokta en yakın merkeze atanır.

13.1 Amaç Fonksiyonu

K-Means, küme içi kareli uzaklıkların toplamını (inertia / WCSS) minimize eder:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2,$$

burada $\boldsymbol{\mu}_k$, C_k kümesinin merkezidir. Bu problemi kesin çözmek NP-zordur; bu yüzden *Lloyd algoritması* ile yinelemeli olarak çözeriz.

13.2 Lloyd Algoritması

İki adım, yakınsayana dek tekrarlanır — bu aslında bir tür “koordinat inişi” optimizasyonudur:

Adım

Yapılan

1. **Atama** Her noktayı en yakın merkeze ata (merkezler sabit).

2. **Güncelleme** Her merkezi, atanan noktaların ortalaması yap (atamalar sabit).

```

1 import numpy as np
2
3 class KMeansScratch:
4     def __init__(self, k=3, max_iters=100, seed=42):
5         self.k, self.max_iters, self.seed = k, max_iters, seed
6
7     def fit(self, X):
8         rng = np.random.default_rng(self.seed)
9         # merkezleri k rastgele veri noktası olarak başlat
10        idx = rng.choice(len(X), self.k, replace=False)
11        self.centroids = X[idx].copy()
12
13        for _ in range(self.max_iters):
14            # Adım 1: her noktayı en yakın merkeze ata

```

```

15     dists = np.linalg.norm(X[:, None] - self.centroids, axis=2)
16     labels = np.argmin(dists, axis=1)
17
18     # Adım 2: merkezleri atanmış noktaların ortalaması olarak yeniden hesapla
19     new_centroids = np.array([
20         X[labels == j].mean(axis=0) if np.any(labels == j)
21         else self.centroids[j]
22         for j in range(self.k)
23     ])
24     if np.allclose(new_centroids, self.centroids):
25         break # yakınsadı
26     self.centroids = new_centroids
27     self.labels_ = labels
28     return self

```

Dikkat

K-Means başlangıç merkezlerine *duyarlıdır*: kötü başlangıç kötü kümeye yakınsayabilir. k-means++ başlatması bu riski büyük ölçüde azaltır (scikit-learn varsayımı budur). Ayrıca *k'yi önceden bilmen gerekir*; doğru *k* için “dirsek yöntemi” (inertia’nın kırıldığı nokta) veya “silhouette skoru” kullanılır.

```

1 from sklearn.cluster import KMeans
2
3 km = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=42)
4 km.fit(X)
5 print(km.inertia_) # küme içi kareler toplamı (J)
6 print(km.cluster_centers_)

```

14. Temel Bileşen Analizi (PCA)

PCA, çok boyutlu veriyi, *bilginin çoğunu koruyarak* daha az boyuta indirgeyen bir tekniktir. Fikir: verinin en çok *yayıldığı* (en yüksek varyanslı) yönleri bul; bu yönler verinin “en önemli” eksenleridir. Bu yönlere *temel bileşenler* denir.

14.1 Matematiksel Türetme

Veri $\mathbf{X}$ 'i önce *merkezle* (her özellikten ortalamasını çıkar). Merkezlenmiş veri için kovaryans matrisi:

$$\Sigma = \frac{1}{m} \mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{n \times n}.$$

Amacımız, üzerine izdüşürüldüğünde varyansı maksimize eden birim yön $\mathbf{v}$ 'yi bulmaktır. İzdüşümün varyansı $\mathbf{v}^\top \Sigma \mathbf{v}$ 'dir.

Türetme

$\|v\| = 1$ kısıtı altında $v^\top \Sigma v$ 'yi maksimize et. Lagrange çarpanı λ ile:

$$\mathcal{L}(v, \lambda) = v^\top \Sigma v - \lambda(v^\top v - 1).$$

v 'ye göre türev alıp sifıra eşitleyince:

$$2\Sigma v - 2\lambda v = 0 \implies \boxed{\Sigma v = \lambda v}.$$

Bu tam olarak bir *özdeğer denklemidir*! Yani en çok varyansa sahip yön, kovaryans matrisinin *en büyük özdeğerine* karşılık gelen özvektördür. Üstelik o yöndeki varyans $v^\top \Sigma v = \lambda$ 'ya eşittir — özdeğer, doğrudan o bileşenin açıkladığı varyanstır.

Böylece PCA'nın reçetesi: kovaryans matrisinin özvektörlerini bul, özdeğerlere göre büyükten küçüğe sırala, en büyük d tanesini al — bunlar aradığın d temel bileşendir.

```
1 import numpy as np
2
3 class PCAScratch:
4     def __init__(self, n_components):
5         self.n_components = n_components
6
7     def fit(self, X):
8         self.mean_ = X.mean(axis=0)
9         X_centered = X - self.mean_           # veriyi merkezle
10        cov = np.cov(X_centered, rowvar=False) # kovaryans matrisi (n, n)
11        eigvals, eigvecs = np.linalg.eigh(cov)  # simetrik -> eigh kullan
12        # özvektörleri özdeğere göre azalan sırada sırala
13        order = np.argsort(eigvals)[::-1]
14        self.components_ = eigvecs[:, order[:self.n_components]]
15        self.explained_variance_ = eigvals[order[:self.n_components]]
16        return self
17
18    def transform(self, X):
19        return (X - self.mean_) @ self.components_ # bileşenlere izdüşür
```

```
1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 X_scaled = StandardScaler().fit_transform(X) # PCA ölçeklenmiş veri gerektirir
5 pca = PCA(n_components=2)
6 X_reduced = pca.fit_transform(X_scaled)
7 print(pca.explained_variance_ratio_)         # korunan varyans oranı
```

İpucu

PCA *ölçeğe duyarlıdır*: büyük değerli özellikler varyansa hâkim olur. Bu yüzden PCA'dan önce mutlaka standartlaştır. `explained_variance_ratio_` toplamı, kaç bileşenle bilginin ne kadarını koruduğunu söyler — genelde %95 varyansı koruyan bileşen sayısı seçilir.

KISIM 6

Sinir Ağları ve Derin Öğrenme

15. Perceptron'dan Çok Katmanlı Ağlara

Sinir ağları, basit hesap birimlerini (nöronlar) katmanlar halinde birleştirerek son derece karmaşık fonksiyonları öğrenebilen modellerdir. Beynin nöronlarından ilham alır ama özünde saf matematiktir: matris çarpımları ve doğrusal olmayan fonksiyonlar.

15.1 Yapay Nöron

Tek bir nöron, tıpkı lojistik regresyon gibi çalışır: girdilerin ağırlıklı toplamını alır, bir *aktivasyon fonksiyonundan* geçirir:

$$a = g\left(\sum_j w_j x_j + b\right) = g(\mathbf{w}^\top \mathbf{x} + b).$$

Tek nöron yalnızca *doğrusal* sınırlar çizebilir. Gücün sırrı, nöronları katmanlar halinde yığıp aralarına *doğrusal olmayan* aktivasyonlar koymaktır — bu, ağın doğrusal olmayan fonksiyonları temsil etmesini sağlar.

15.2 Aktivasyon Fonksiyonları

Doğrusal olmama şarttır: aktivasyon olmadan, kaç katman olursa olsun ağ tek bir doğrusal dönüşüme çöker. Yaygın seçenekler:

Fonksiyon	$g(z)$	Özellik
Sigmoid	$\frac{1}{1+e^{-z}}$	(0,1); doyumlukta gradyan kaybolur.
Tanh	$\tanh(z)$	(-1,1); sıfır merkezli.
ReLU	$\max(0, z)$	En yaygın; hızlı, gradyan kaybını azaltır.
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	Çok sınıflı çıkış (olasılıklar).

Bilgi

Evrensel yaklaşım teoremi: Yeterli sayıda nörona sahip tek gizli katmanlı bir ağ, herhangi bir sürekli fonksiyonu istenen doğrulukta yaklaşık temsil edebilir. Ama “yapabilir” ile “öğrenebilir” farklıdır — pratikte derin (çok katmanlı) ağlar, aynı işi çok daha az nöronla ve daha kolay öğrenerek yapar. Derinliğin gücü buradadır.

15.3 İleri Besleme (Forward Propagation)

Çok katmanlı bir ağda bilgi, girişten çıkışa katman katman akar. l . katman için (matris biçiminde, tüm örnekler birden):

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}, \quad \mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]}),$$

burada $\mathbf{A}^{[0]} = \mathbf{X}$ (giriş), $\mathbf{A}^{[L]} = \hat{\mathbf{Y}}$ (çıkış). Her katman bir doğrusal dönüşüm + doğrusal olmayan aktivasyondan ibarettir.

```

1 import numpy as np
2
3 def relu(Z):
4     return np.maximum(0, Z)
5
6 def softmax(Z):
7     Z = Z - Z.max(axis=1, keepdims=True)      # sayısal kararlılık
8     exp = np.exp(Z)
9     return exp / exp.sum(axis=1, keepdims=True)
10
11 def forward(X, params):
12     """İki katmanlı ağ: X -> ReLU -> softmax."""
13     Z1 = X @ params['W1'] + params['b1']
14     A1 = relu(Z1)
15     Z2 = A1 @ params['W2'] + params['b2']
16     A2 = softmax(Z2)
17     cache = (X, Z1, A1, Z2, A2)                # backprop için saklandı
18     return A2, cache

```

16. Geri Yayılım (Backpropagation)

Geri yayılım, sinir ağlarının nasıl öğrendiğinin kalbidir: kaybın, ağdaki *her ağırlığa* göre gradyanını verimli biçimde hesaplar. Özü, Kısım 1'deki *zincir kuralıdır* — hata, çıkıştan girişe doğru katman katman “geri yayılır”. Bu bölüm, iki katmanlı bir ağ için gradyanları baştan sona türetir.

16.1 Neden Zincir Kuralı?

Kayıp J , çıkışa; çıkış, son katman ağırlıklarına; o ağırlıklar bir önceki katmanın çıkışına... bağlıdır. Bir ara ağırlığın kaybı nasıl etkilediğini bulmak için zincir kuralıyla bu bağımlılık zincirini çarparız:

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \frac{\partial J}{\partial \mathbf{A}^{[l]}} \cdot \frac{\partial \mathbf{A}^{[l]}}{\partial \mathbf{Z}^{[l]}} \cdot \frac{\partial \mathbf{Z}^{[l]}}{\partial \mathbf{W}^{[l]}}.$$

16.2 Tam Türetme (Softmax + Çapraz Entropi)

Türetme

Ağ: $\mathbf{X} \rightarrow \mathbf{Z}^{[1]} \rightarrow \mathbf{A}^{[1]} = \text{ReLU} \rightarrow \mathbf{Z}^{[2]} \rightarrow \mathbf{A}^{[2]} = \text{softmax}$, kayıp çapraz entropi. Türetmeyi çıkıştan geriye yaparız.

(1) Çıkış katmanı. Softmax + çapraz entropinin birlikte türevi olağanüstü sadeleşir (bu ikilinin birlikte kullanılmasının nedeni budur):

$$\frac{\partial J}{\partial \mathbf{Z}^{[2]}} = \mathbf{A}^{[2]} - \mathbf{Y} \equiv \boldsymbol{\delta}^{[2]}.$$

Buradan son katman gradyanları:

$$\frac{\partial J}{\partial \mathbf{W}^{[2]}} = \frac{1}{m} (\mathbf{A}^{[1]})^\top \boldsymbol{\delta}^{[2]}, \quad \frac{\partial J}{\partial \mathbf{b}^{[2]}} = \frac{1}{m} \sum_i \delta_i^{[2]}.$$

(2) Hatayı geri yay. Gizli katmanın hatası, çıkış hatasının ağırlıklarla geri taşıyıp ReLU türeviyle çarpılmasıdır:

$$\boldsymbol{\delta}^{[1]} = (\boldsymbol{\delta}^{[2]} (\mathbf{W}^{[2]})^\top) \odot g'(\mathbf{Z}^{[1]}), \quad g'(z) = \mathbb{I}[z > 0] \quad (\text{ReLU türevi}).$$

Burada $\odot$ eleman bazlı çarpımdır.

(3) İlk katman gradyanları. Aynı kalıp:

$$\frac{\partial J}{\partial \mathbf{W}^{[1]}} = \frac{1}{m} \mathbf{X}^\top \boldsymbol{\delta}^{[1]}, \quad \frac{\partial J}{\partial \mathbf{b}^{[1]}} = \frac{1}{m} \sum_i \delta_i^{[1]}.$$

```

1 def backward(Y, cache, params):
2     """Yukarıdaki iki katmanlı ağ için geri yayılım.
3     Y: one-hot kodlanmış gerçek etiketler (m, n_classes)."""
4     X, Z1, A1, Z2, A2 = cache
5     m = X.shape[0]
6     grads = {}
7
8     # (1) Çıkış katmanı hatası: softmax + çapraz entropi => A2 - Y
9     dZ2 = A2 - Y # (m, n_classes)
10    grads['W2'] = (A1.T @ dZ2) / m
11    grads['b2'] = dZ2.sum(axis=0, keepdims=True) / m
12
13    # (2) Hatayı gizli katmana geri yay (ReLU türevi)
14    dA1 = dZ2 @ params['W2'].T
15    dZ1 = dA1 * (Z1 > 0) # ReLU'(z) = 1[z>0]
16
17    # (3) İlk katman gradyanları
18    grads['W1'] = (X.T @ dZ1) / m
19    grads['b1'] = dZ1.sum(axis=0, keepdims=True) / m
20    return grads
21
22 def update(params, grads, lr):
23     for key in params:
24         params[key] -= lr * grads[key] # gradyan inişi adımı
25     return params

```

Bilgi

Backprop'un "geri yayma" kalıbına dikkat et: her katmanın gradyanı $\mathbf{A}^\top \boldsymbol{\delta}$ biçimindedir ve hata $\boldsymbol{\delta}$, bir üst katmandan $\mathbf{W}^\top$ ile geri taşınır. Bu tekrarlayan yapı, PyTorch/TensorFlow gibi kütüphanelerin *otomatik türev* mekanizmasının temelidir — sen ileri geçişi yaz, onlar bu zinciri otomatik kurar.

16.3 Kaybolan/Patlayan Gradyan

Derin ağlarda gradyan, katmanlar boyunca çarpıla çarpıla geri yayılır. Çarpanlar sürekli < 1 ise gradyan katman katman *küçülüp kaybolur* (vanishing); > 1 ise *patlar* (exploding). Sigmoid/tanh doygunlukta gradyanı sıfıra iter — bu yüzden derin ağlarda ReLU tercih edilir. Ayrıca dikkatli ağırlık başlatma (Xavier/He), batch normalization ve residual bağlantılar bu sorunu hafifletir.

17. Optimizasyon Algoritmaları

Düz gradient descent, sinir ağlarını eğitmek için genelde yavaştır ve dar vadilerde salınır. Modern optimize ediciler, geçmiş gradyanları kullanarak öğrenmeyi hızlandırır ve kararlı hale getirir.

17.1 Momentum

Momentum, gradyanların bir *hareketli ortalamasını* (hız) tutar; böylece tutarlı yönlerde ivmelenir, gürültülü salınımları söndürür — yokuş aşağı yuvarlanan bir topa benzer:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + (1 - \beta) \nabla J, \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{v}.$$

17.2 Adam: En Yaygın Optimize Edici

Adam (Adaptive Moment Estimation), momentum ile parametre-başına uyarlamalı öğrenme oranını birleştirir. Gradyanın hem birinci momentini (ortalama, $\mathbf{m}$) hem ikinci momentini (varyans, $\mathbf{v}$) tutar:

$$\begin{aligned} \mathbf{m} &\leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \mathbf{g}, & \mathbf{v} &\leftarrow \beta_2 \mathbf{v} + (1 - \beta_2) \mathbf{g}^2, \\ \hat{\mathbf{m}} &= \frac{\mathbf{m}}{1 - \beta_1^t}, & \hat{\mathbf{v}} &= \frac{\mathbf{v}}{1 - \beta_2^t}, & \mathbf{w} &\leftarrow \mathbf{w} - \eta \frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}} + \epsilon}}. \end{aligned}$$

$\hat{\mathbf{m}}, \hat{\mathbf{v}}$ "bias düzeltilmesidir" (başlangıçta sıfıra yakın olmayı telafi eder). Adam çoğu problemde iyi çalışır ve az ayar gerektirir — bu yüzden derin öğrenmede varsayılan tercihtir.

Optimize edici	Fikir	Not
SGD	Saf gradyan adımı	Basit, ayar ister
Momentum	Geçmiş gradyanla ivmelen	Salınımı azaltır
RMSProp	Uyarlamalı öğrenme oranı	RNN'lerde iyi
Adam	Momentum + RMSProp	En yaygın varsayılan

18. Derin Öğrenme Mimarileri ve Pratik

Tam bağlı ağlar (MLP) her şeyi öğrenebilir ama görüntü, dizi gibi *yapılı* veride verimsizdir. Özel mimariler, verinin yapısını mimariye gömer.

18.1 Evrişimli Ağlar (CNN)

CNN'ler görüntü için tasarlanmıştır. Tam bağlantı yerine, küçük *filtreleri* (kernel) görüntü üzerinde kaydırarak yerel örüntüleri (kenar, köşe, doku) yakalar. Üç temel fikir: **yerel bağlantı** (her nöron sadece küçük bir bölgeye bakar), **ağırlık paylaşımı** (aynı filtre her yerde kullanılır — çok az parametre) ve **havuzlama** (pooling ile boyut indirgeme). Katmanlar derinleştikçe basit örüntülerden karmaşık nesnelere doğru hiyerarşik özellikler öğrenilir.

18.2 Tekrarlayan Ağlar (RNN) ve Transformer

RNN'ler dizisel veri (metin, zaman serisi, ses) içindir: bir *gizli durum* tutarak geçmiş bilgiyi taşırlar. Uzun bağımlılıklarda kaybolan gradyan sorunu yaşarlar; **LSTM** ve **GRU** kapı (gate) mekanizmalarıyla bunu çözer. Günümüzde metin ve giderek daha çok alanda, dizinin tüm parçalarına aynı anda bakan *dikkat* (attention) mekanizmasına dayalı **Transformer** mimarisi baskındır — büyük dil modellerinin (LLM) temelidir.

```

1 # Modern derin öğrenme sıfırdan değil, framework'lerle yapılır.
2 # Yukarıda türettiğimiz iki katmanlı ağın minimal bir PyTorch örneği:
3 import torch
4 import torch.nn as nn
5
6 model = nn.Sequential(
7     nn.Linear(784, 128),      # giriş -> gizli
8     nn.ReLU(),
9     nn.Linear(128, 10)       # gizli -> çıkış (10 sınıf)
10 )
11 criterion = nn.CrossEntropyLoss()      # softmax + çapraz entropi
12 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
13
14 for epoch in range(n_epochs):
15     for xb, yb in dataloader:           # mini-gruplar
16         optimizer.zero_grad()           # gradyanları sıfırla
17         preds = model(xb)                # ileri geçiş
18         loss = criterion(preds, yb)
19         loss.backward()                   # backprop (otomatik!)
20         optimizer.step()                  # Adam güncellemesi

```

İpucu

Otomatik türev sayesinde artık gradyanları elle yazmıyoruz — `loss.backward()` tüm zinciri otomatik hesaplar. Ama bu rehberde backprop'u elle üretmemizin nedeni tam da bu: “sihrin” altında sadece zincir kuralı olduğunu bilmek, ağırları hata ayıklamak, yeni katmanlar tasarlamak ve neyin neden çalışmadığını anlamak için vazgeçilmezdir.

19. Kapanış: Bir Bütün Olarak Makine Öğrenmesi

Bu rehber boyunca gördük ki makine öğrenmesi, üç matematiksel temelin kaynaşmasıdır:

Temel	Rolü
Lineer cebir	Veriyi ve modeli temsil eder (vektör/matris işlemleri).
Kalkülüs & optimizasyon	Kayıbı minimize ederek modeli eğitir (gradient descent).
Olasılık & istatistik	Belirsizliği modeller, kayıpları türetir (MLE, cross-entropy).

Aynı fikirlerin tekrar tekrar döndüğüne dikkat et: gradyan hem lineer regresyonda, hem lojistik regresyonda, hem sinir ağında *aynı biçimi* alır ($\mathbf{X}^\top(\hat{\mathbf{y}} - \mathbf{y})$). MLE hem MSE'yi hem çapraz entropiyi doğurur. Bir kez bu ortak yapıyı gördüğünde, yeni bir algoritma artık ezberlenecek bir formül değil, aynı temaların bir varyasyonu olur.

19.1 Bir Model Nasıl Seçilir?

Durum	Önerilen model
Az veri, yorumlanabilirlik	Lineer/lojistik regresyon, karar ağacı
Tablo veri, en iyi performans	Gradient boosting (XGBoost/LightGBM)
Sağlam, az ayarlı ilk deneme	Random Forest
Görüntü	CNN (evrişimli ağ)
Metin / dizi	Transformer (veya LSTM)
Etiketsiz, grup bulma	K-Means, PCA

İpucu

Son bir tavsiye: **basit başla**. Her zaman basit bir temel çizgi (baseline) modelle başla — lineer/lojistik regresyon veya random forest. Karmaşık bir modele ancak basit olan yetmediğinde geç. Çoğu zaman iyi *özellik mühendisliği* ve temiz veri, fancy bir modelden çok daha fazla kazandırır. Ve her şeyi bir doğrulama kümesinde ölç — eğitim skoruna asla güvenme. İyi öğrenmeler!